

Excavatortron - Session Handoff

***Purpose of this file.** This is a complete, self-contained brief so a brand-new Claude session can continue building **Excavatortron** with zero context loss. Read it top to bottom, then jump to **§12 Pending / Next steps** and resume. Last updated after commit `4dfcb4c`` (in sync with origin/main).*

0. How to use this document (instructions for the new session)

1. The project lives at C:\Users\leitan\AI-YouTube-Skills (a git clone of GitHub repo Eitanvinokur12345/AI-YouTube-Playlist-Information-Extractor).
2. Use git -C "C:/Users/eitan/AI-YouTube-Skills" ... for git, or note that the Bash tool's working directory may reset to the session outputs dir - always pass absolute paths.
3. Do not re-ask the locked decisions in §4. They were already answered by the user.
4. Honor every rule in §5 Standing constraints - especially: no babysitting, no new paid cost, never commit secrets / make_icon.py / .claude/ / temp files, push automatically, never modify frozen records.
5. The authoritative quality checklist is the user's own spec, preserved verbatim in docs/REFERENCE_SPEC.md (see §14).

1. What Excavatortron is

A self-running, self-improving dashboard that mines a YouTube playlist of AI videos and turns everything it finds - techniques (skills), products (tools), models, MCP connectors, slash commands, tips, and news - into a clean, searchable web dashboard. It runs in the cloud for free and improves itself on a schedule.

Core promise (the prime directive): the owner should not have to intervene and should not have to pay anything beyond what they already have. Free = public-repo GitHub Actions + the Claude Pro/Max subscription token (not paid API billing) + free external API tiers that skip gracefully when absent.

2. Where everything lives

Thing | Location
Local repo clone | `C:\Users\leitan\AI-YouTube-Skills`
GitHub repo | `Eitanvinokur12345/AI-YouTube-Playlist-Information-Extractor`
Dashboard (GitHub Pages) | served from `docs/`, reads `../data/\*.json`
Playlist ID | `PLxtsVIUtYhNo6pY9FrVqVe2xh---Y8rxB`
User's SKILLS folder | `C:\Users\leitan\OneDrive\Desktop\claude skills of eitan`
User's DATA folder (legacy spec) | `C:\Users\leitan\OneDrive\Desktop\AI Skills Data`
Claude **Desktop** MCP config | `C:\Users\leitan\AppData\Roaming\Claude\claude\_desktop\_config.json`
Repo's own skill folders (learning source) | `./skills`, `./other-skills`
The Obsidian "brain" | `./brain/` (committed in repo; mirror to Desktop via local runner)

The user is in Israel (UTC+2 IST winter / UTC+3 IDT summer) - "night Israel time"
work is scheduled for Saturday night UTC.

3. The user's original request & intent (the expansion being executed)

Verbatim intent (paraphrased tightly from the user's own words across sessions):

- (a) Extract everything the video *and its surroundings* have to offer, in large patches - be able to analyze ~100 videos within 2 days at a high level; if that's not feasible, fall back to a once-weekly update at night Israel time.
 - (b) Self-improvement must, each run, return to the user's reference format (the "YouTube Skills Tracker - System Prompt") to check it's all there and improve those elements, and return to the user's skills folder to find ways to improve information extraction and design.
 - (c) Make an Obsidian "brain" with all project info, and tell the user how to give Claude access to Obsidian.
 - (d) Run higher self-improvement iterations in the first week, reinforced by external agents (another cloud/GitHub code agent, possibly discovered from the videos) so that after each self-improvement pass, at least 3 different agents check under different conditions: (1) usability improvement, (2) "cut the bullshit", (3) a deep code-bug researcher.
 - The user's pasted System Prompt (7 tabs + 50 self-check questions) is the reference checklist - adapted to the cloud + dashboard architecture (see §14).
-

4. Locked decisions - DO NOT re-ask (answered via AskUserQuestion)

1. Obsidian = a vault in the repo at brain/ + a Desktop mirror + MCP setup instructions for the user.
 2. Cadence = high-frequency ingest + ONE weekly deep pass at night Israel time.
 3. Trends = NO fixed trend tab. Instead build DYNAMIC TAB DISCOVERY: inspect raw data before routing, detect off-tab recurring "sequences/anecdotes", spawn a new tab with a NEW badge + description for the first week (badge then auto-expires), connected to anecdotes found during the first days of tracking.
 4. Review = Claude reviews FIRST, then a DIFFERENT external engine verifies (free token, graceful-skip). Dimensions = usability/UX (competitor-benchmarked) + "cut the bullshit" + deep code-bug researcher. Higher first-week iteration.
-

5. Standing constraints (MUST follow - quoted where exact)

- "the most important rule is that there will be [no] my intervention and preferably that I don't have to pay anymore" -> no babysitting; no new paid costs; free tiers OK; the external review token is free-tier + graceful-skip.
 - "Ask as many questions as possible so that you can do it well" -> build-time clarifying questions are welcome (but the §4 decisions are already locked).
 - "For the absolute OFFLINE, you cannot use an external connector like PLAYWRIGHT to do this."
 - Never commit secrets. API keys/tokens ONLY in GitHub Actions secrets / local env vars. Never print or commit them.
 - Star = "proven excellent, leave in original form, never change," max 10; never touch FROZEN records (data/stars.json, or starred/locked: true).
 - Push automatically without asking.
 - Output language English only; never edit/translate the SOURCE transcript (use YouTube's text exactly as-is).
 - Never commit: make_icon.py (repo root, untracked), .claude/, and any temp files: C:\Users\leitan_tmp_*.py, C:\Users\leitan_transcript_tmp.jsonl, C:\Users\leitan_refspec_raw.txt, _batch_digest.json. Stage specific paths, never git add -A.
 - Commit trailer: Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com> (LF->CRLF warnings on Windows are harmless).
-

6. Architecture (the cloud pipeline)

```

YouTube playlist
| (local runner OR cloud)
v
[fetch]   src/fetch.py + fetch.yml      -> data/_pending/*.json (+ transcript, 80k chars)
v
[analyze] CLAUDE.md + analyze.yml      -> skills/tools/models/connectors/commands/tips
|                                           commits + PUSHES after EACH video (crash-safe)
v
[news]    src/news.py + news.yml       -> daily/weekly/monthly (+ web) news feeds
v
[improve] IMPROVE.md + improve.yml     -> tidy, calibrate, self-check, dynamic tabs (DEEP PASS)
v
[review]  REVIEW.md + review.yml       -> data/review_findings.json (3 agents)
|                                           + codeql.yml (free static analysis, the automated deep-code arm)
v
docs/    --> GitHub Pages --> live dashboard (vanilla JS, reads ../data/*.json)

```

Engines are Markdown specs that Claude follows in the cloud - they, not the Python, are the live logic:

- CLAUDE.md - analyze engine.
- IMPROVE.md - deep-pass engine.
- REVIEW.md - 3-agent review engine.
- docs/REFERENCE_SPEC.md - the user's spec (Part A verbatim) + cloud mapping (Part B) + the 50 self-check questions (Part C). Referenced by config.reference_self_check.spec_path.

Dead/legacy code (kept for reference, NOT the live path): src/process_video.py, src/analyze_batch.py. The live analysis runs from CLAUDE.md.

Secrets (names only - never values):

- YOUTUBE_API_KEY - fetch.
- CLAUDE_CODE_OAUTH_TOKEN_REAL - the Pro/Max subscription token for all Claude steps (renew ~yearly with claude setup-token).
- EXTERNAL_REVIEW_API_KEY - Gemini free tier for the external review second opinion (optional; graceful-skip if absent).

7. Cadence (crons, all UTC)

- Ingest (high-frequency): analyze.yml - 0 */3 * * * (every 3h, 50 oldest-first) + */30 * * * * (catch-up "sprint", no-ops unless catch-up active). fetch.yml runs frequently too. Throughput target: ~100 videos / 48h.
- Weekly DEEP PASS at night Israel time:
- improve.yml - 0 20 * * 6 (Sat 20:00 UTC, the "tidy" lead-in).
- review.yml - 0 23 * * 6 (Sat 23:00 UTC ~ Sun ~02:00 Israel, the "critique").
- 3-hour stagger so they never fight the git tree.
- First-week intensive (nightly): improve 0 20 * * 0-5, review 0 23 * * 0-5 (both guarded - no-op after week 1). The window is anchored once in data/review_state.json (first_run_at) and shared by both stages. config.cadence.first_week.started_at (currently null) overrides the anchor.
- CodeQL: codeql.yml on push to code paths + 0 22 * * 6 weekly + dispatch.
- Catch-up mode: src/fetch.py flips data/catch_up.json active on a big burst; analyze switches to large, newest-first batches drained by the */30 sprint until empty, then auto-returns to normal.

8. Key systems

- Skills vs Tools (the #1 rule): **skills/techniques** (something you DO) and **tools/products** (something that EXISTS) are SEPARATE records and SEPARATE tabs/files (skills.json vs tools.json). Models are a subset of tools mirrored into models.json. Connectors get their own connectors.json.
- Dynamic tabs: analyze captures off-tab anecdotes -> data/tab_candidates.json {theme,label,note,video_id,source_url,ts}; improve Step 7 clusters them -> data/extra_tabs.json with created_at, badge_until (= created_at + new_badge_days, default 7), and a description from the anecdotes; the dashboard renders them generically with a NEW badge that auto-expires (honors badge_until, falls back to created_at+7d).
- Reference self-check loop: improve Step 7c (Module 9) re-answers the 50 questions from REFERENCE_SPEC.md -> data/self_check.json {ran_at,score,total,improvements_logged, results[{n,question,answer,evidence}]} + opens data/improvement_tasks.json {tasks[{n,question,fix,kind,status,created_at}]} (kinds: safe_auto/needs_approval/engine_followup); improve Step 1b auto-applies safe tasks next run. Loop closes.
- 3-agent review: Claude first (REVIEW.md) -> external Gemini second opinion (src/external_review.py, graceful-skip) + CodeQL. Output data/review_findings.json {generated_at,mode,reviewers{claude,external,codeql},scores{usability,cut_the_bullshit,deep_code_bugs,overall},benchmark{competitors,we_do_better,they_do_better,borrow_next}, findings[...],top_actions,history}. Read-only on content; may write ui_change / skills_folder_learning suggestions for the user to approve. First-week intensive.
- Skills-folder learning: improve Module 8 / Step 6b reads ./skills + ./other-skills and writes skills_folder_learning suggestions (target analyze or dashboard).
- Stars / freezing: max 10 starred; frozen records (data/stars.json or starred/locked: true) are NEVER modified/merged/rescored/deleted.

9. File inventory (status)

Engines (root): CLAUDE.md (analyze), IMPROVE.md (deep pass, has Step 1b/6b/7/7c), REVIEW.md (3 agents), docs/REFERENCE_SPEC.md (spec + 50 Qs). Also README.md, PIPELINE.md.

Workflows (.github/workflows/): fetch.yml, analyze.yml (step renamed to "Analyze pending videos (skills, tools, models, connectors, commands)"; prompt updated for skills/tools split + tab_candidates), news.yml, improve.yml (now weekly + first-week plan step), review.yml, codeql.yml.

Code (src/): fetch.py (live), news.py (live), external_review.py (live, stdlib-only, graceful-skip), merge_dupes.py; dead: process_video.py, analyze_batch.py.

Dashboard (docs/): index.html (+CSS, has .bench/severity/kind badge styles), dashboard.js (renders all tabs incl. self-check, fix tasks, review findings, dynamic tabs with tabsNew() badge-expiry), sw.js (shell cache v5), manifest.webmanifest, icon.png.

Data (data/): content (skills/tools/models/connectors/commands/tips/agent_catalog), news (daily/weekly/monthly[_web]_news, web_news_store, feeds_health), state (status, index, catch_up, _pending/, processed/), self-improve/review (self_check, improvement_tasks, review_findings, tab_candidates, extra_tabs; created-on-first-run: health, improvement_suggestions, approvals, improvement_audit, review_state, stars), housekeeping (deleted_skills, merge_log).

Brain (brain/): an Obsidian vault - see §11 for which notes exist / remain.

MCP server (mcp_server/): server.py, claude_desktop_config.example.json - the

offline MCP tools (approve/dismiss suggestions, star/unstar, run_improve, dismiss_dynamic_tab, etc.).

Sync (sync/): PowerShell helpers (setup-sync.ps1, sync-skills.ps1, open-dashboard-local.ps1, create-shortcut.ps1).

config.json - central config; key blocks in §13/§14.

10. Gotchas & hard-won lessons

- Bash tool cwd resets to the session outputs dir between turns/continuations - always use absolute paths or git -C.
- Windows MAX_PATH (260 chars): native Python open() fails on long paths where git-bash cp/lis succeed. Fix: cp the file to a short path (e.g. C:\Users\leitan_x.tmp) then read.
- Python doesn't grok MSYS /c/ paths; use C:\... or C:/... for native Python.
- Bash tool sandboxes to the project dir; use dangerouslyDisableSandbox: true to read outside it.
- Python cp1252 stdout crashes on Unicode -> sys.stdout.reconfigure(encoding="utf-8").
- git: git pull --rebase can replay old commits and cause spurious add/add conflicts when a local merge commit already incorporates origin history - use git merge origin/main in that case. (Workflows use git pull --rebase --autostash || true then git push || echo.)
- Never git add -A (would catch make_icon.py). Stage explicit paths.

11. What's DONE (with commits)

Tasks #1-#16 complete. Recent commits:

- e2d4fec - engine: split skills/tools, rewrite analyze spec, wire expanded scope.
- db97327 - improve: reference self-check (50Q), skills-folder learning, dynamic tabs.
- 408c907 - review: 3-agent quality gate (Claude-first, external, CodeQL).
- 4dfcb4c - dashboard surfaces self-check + 3-agent review; dynamic-tab badge auto-expiry; cadence weekly deep pass at night Israel time (improve->weekly+first-week; analyze wording fix).

Brain vault - notes already written (9): README.md, Excavatortron Brain.md (home/MOC), Architecture.md, Engines.md, Skills vs Tools.md, Tabs.md, Data Files.md, Cadence.md, Pipeline - Fetch.md. (brain/ is currently UNTRACKED - not yet committed.)

12. PENDING / Next steps (resume here)

Task #17 (in progress) - finish the Obsidian brain vault

Write the remaining notes referenced by existing wikilinks (so none dangle), then commit brain/ (specific path; NOT make_icon.py). Remaining notes to create in brain/:

- Pipeline - Analyze.md, Pipeline - News.md, Pipeline - Improve.md, Pipeline - Review.md
 - Dynamic Tabs.md, Reference Self-Check.md, Three-Agent Review.md, Stars and Freezing.md
 - Self-Improvement Loop.md, Config Reference.md, Operations and Setup.md
 - Obsidian Access (MCP).md - must tell the user how to give Claude access to Obsidian: (Option A, simplest) an MCP server pointed at the vault path in claude_desktop_config.json; (Option B) Obsidian Local REST API community plugin + an Obsidian MCP server using its API key (key stays local, never in repo). Restart Claude Desktop after editing the config.
- Flag that community MCP package names evolve - tell the user to verify the current package.

- Glossary.md

Then: in the home note, confirm the Desktop-mirror story (the local runner syncs brain/->Desktop).

Task #6 (pending) - local automated fetch runner (Windows)

A one-command Task Scheduler setup that: `git pull -> python -m src.fetch (residential IP) -> commit + push the new data/_pending/* -> also sync brain/->Desktop and read the user's skills folder. Consider enhancing fetch.py to also capture top comments + full description (part of "everything the surroundings offer")`. Keep it free, no babysitting.

Task #7 (pending) - final commit/push + content + cleanup + user instructions

- Create the missing 10th technique seedance-ugc SKILL.md (referenced but absent).
- Re-check cloud commits, then commit + push everything (specific paths).
- Delete temp files: `C:\Users\leitan\tmp_*.py, _transcript_tmp.jsonl, _refspec_raw.txt`.
- Ensure `make_icon.py` and `.claude/` are NOT committed.
- Write consolidated user instructions: GitHub secrets setup (the 3 keys), enabling GitHub Pages, the external review token, Task Scheduler local runner, and Obsidian access.

13. Git state & verification

- Latest local commit: `4dfcb4c`; in sync with `origin/main` (0 ahead / 0 behind).
- Verify: `git -C "C:/Users/eitan/AI-YouTube-Skills" fetch origin main` then `git -C ... rev-list --left-right --count origin/main...HEAD -> expect 0 0`.
- Untracked (expected): `brain/` (commit when ready), `make_icon.py` (NEVER commit).

Key `config.json` blocks: `dynamic_tabs` (`new_badge_days:7`, `candidates_file`, `reserved_tab_ids`), `reference_self_check` (`spec_path`, `questions:50`, `results_file`, `tasks_file`, `auto_fix_next_run:true`), `skills_folder_learning` (`read_paths`), `review` (`dimensions`, `claude_first_then_external`, `usability.competitors`, `external_engine` {`provider gemini`, `secret_name`, `graceful_skip_if_absent`}, `first_week_intensive`, `findings_file`), `cadence` (`mode:high_freq_plus_weekly_deep`, `weekly_deep_pass`, `first_week`, `throughput_target`), `extraction` (`exhaustive:true`, `transcript_chars:80000`, `capture_tab_candidates`, `tab_candidates_file`).

14. The reference spec (the user's checklist)

Preserved verbatim in `docs/REFERENCE_SPEC.md`:

- Part A - the user's original "YouTube Skills Tracker - System Prompt": 7 tabs (1 Skills Library, 2 Models Ranking with [1st][2nd][3rd] podium, 3 Skills Improvement, 4 Tips & Commands, 5 News Feed daily/weekly/monthly, 6 Connectors with display format + connect instructions, 7 Trend Recognition), the RUN REPORT box, the SELF-IMPROVEMENT SYSTEM, and the MCP tool requests.
- Part B - cloud-architecture mapping (translation table + 6 deltas): `48h->every-3h+spring`; `0.5s wait->rate_limit_seconds`; `30-min cap->per-video-commit`; `last_run.json->status.json`; `seven-tabs->six-core+dynamic`; `deltas = skills/tools split`, `trends->dynamic tabs`, `deep extraction (80k chars)`, `batch speed`, `3-agent review`, `stars/freezing`.
- Part C - the 50 self-check questions verbatim, each annotated with its cloud verification + the data file that proves it.

The deep-pass self-check (IMPROVE.md Module 9) answers these every run; the dashboard's Self-Improvement tab shows "Self-check score X/50 - Y improvements logged" + the open gaps.

15. Quick-start for the new session

```
R="C:/Users/eitan/AI-YouTube-Skills"
git -C "$R" fetch origin main && git -C "$R" rev-list --left-right --count origin/main...HEAD # expect
```

```
0 0
git -C "$R" log --oneline -5
ls "$R/brain/"          # see which brain notes exist; finish the rest (see §12)
```

Then continue task #17 (finish brain/), then #6 (local runner), then #7 (seedance-ugc SKILL.md + cleanup + user instructions). Follow §5 at all times.

End of handoff.